

MAC 0417/5768 - Visão e Processamento de Imagens

EP1 – Controle de Qualidade Automatizado

Prof. Ronaldo Fumio Hashimoto

Abril de 2026

1 Introdução

Em ambientes de produção de alta precisão, como na indústria aeroespacial ou na fabricação de dispositivos médicos, qualquer falha na contagem ou separação de componentes pode comprometer não apenas o funcionamento de um sistema, mas também colocar vidas em risco. Por isso, sistemas automatizados de inspeção visual são amplamente utilizados para garantir que todos os elementos estejam presentes, corretamente identificados e organizados.

Neste cenário, uma câmera capturou uma imagem da esteira de triagem final em uma linha de montagem automatizada. Nela, há uma mistura de parafusos, porcas e arruelas. Veja um exemplo na Fig. 1.



Figura 1: Na imagem há parafusos, porcas e arruelas sobre uma superfície plana.

Os parafusos, as porcas e as arruelas precisam ser:

1. Identificados corretamente;
2. Contados com precisão;
3. Classificados ou removidos conforme critérios técnicos.

Seu papel é desenvolver uma solução computacional baseada em visão computacional clássica, empregando binarização, rotulação de componentes conexos e métricas geométricas para analisar a imagem fornecida.

2 Objetivo

Utilizando ferramentas de processamento de imagem, desenvolva um pipeline que:

1. Conte quantos objetos há na imagem;
2. Classifique e separe os objetos em 2 categorias (parafusos $\times$ porcas e arruelas);
3. Garante que, ao final, restem apenas os parafusos – prontos para a próxima etapa da montagem.

3 Relevância

Esse tipo de análise automatizada é base de sistemas modernos de controle de qualidade por visão computacional, e representa um problema real enfrentado em linhas de produção modernas. A capacidade de desenvolver algoritmos robustos para tarefas desse tipo é cada vez mais valorizada em setores industriais que buscam eficiência, segurança e rastreabilidade.

4 Tarefa

A sua tarefa é, a partir de uma imagem (como a da Fig. 1), desenvolva um programa em Python que:

1. Aplique limiarização para obter imagens binárias.
2. Aplique convolução em imagens binárias.
3. Utilize *Union-Find* com vizinhança 8 para rotular e contar os componentes conexos em imagens binárias.
4. Classifique e separe os objetos em três categorias (parafusos, porcas e arruelas).

5 Requisitos e Funções Esperadas

O seu código de ser escrito usando a linguagem *Python*. Além disso, as únicas bibliotecas que você pode usar são: (1) *Numpy*; (2) *Matplotlib* e (3) *gdown*. Mais adiante, será explicado o uso da biblioteca *gdown*. Não use outras bibliotecas! Ou seja, no seu programa, você terá a seguinte linhas de código:

Listing 1: Use somente essas bibliotecas/módulos

```
import gdown
import numpy as np
import matplotlib.pyplot as plt
```

6 Entrega

Para entrega desse exercício, crie um *Google Colab Jupyter Notebook*. Esse *notebook* deve ser em formato de um relatório mostrando seu sistema de controle de qualidade automatizado, dividido em seções e mostrando de forma intercalada **textos** e **códigos**.

7 Fazendo *download* da imagem no *Google Colab*

A biblioteca *gdown* serve para fazer o *download* de um arquivo a partir de um *link* de compartilhamento público do *Google Drive*. Em particular, use o código abaixo para colocar o arquivo

ep1_grayscale_8bit.png da imagem mostrada na Fig. 1 no ambiente do *Google Colab*.

Listing 2: Download da imagem

```
url = 'https://drive.google.com/file/d/1pTILSYmc2UEXNMu0hXyhA91Z0DFxaeY9/view?usp=sharing'
output_file = gdown.download(url, quiet=True, fuzzy=True)
print(f"File {output_file} downloaded")
```

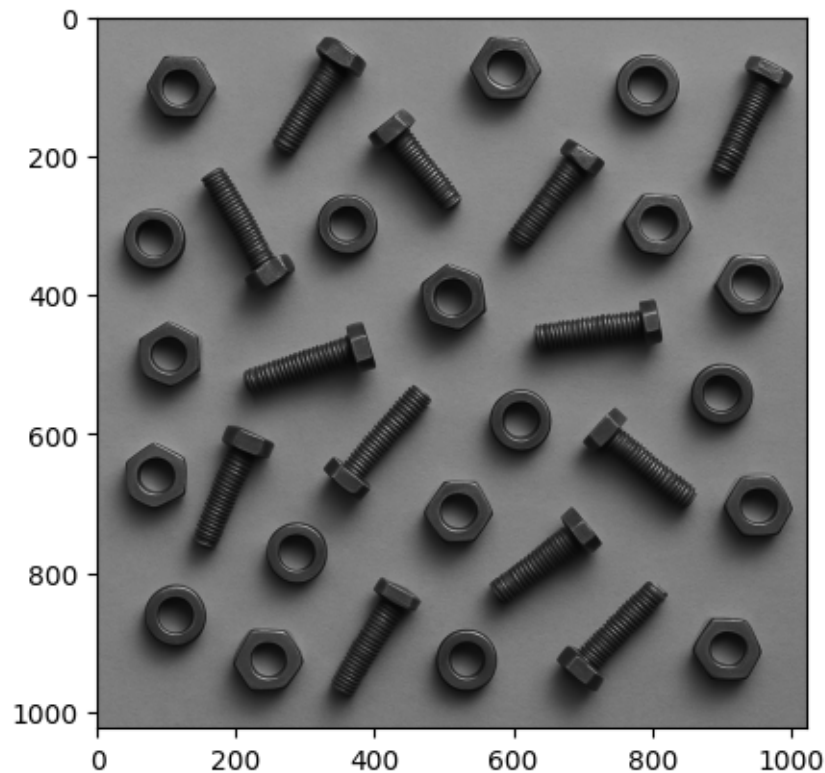
7.1 Função load_image

Faça uma função de nome `load_image` que recebe um nome de arquivo que contém uma imagem e devolve um *array Numpy* contendo a imagem. Essa matriz deve conter valores inteiros. Escale os valores de intensidade para que o mínimo seja 0 e o máximo, 255.

Listing 3: Carregando a imagem em uma matriz

```
input_img = load_image (output_file)

# Exibe a imagem
plt.imshow(input_img, cmap='gray')
plt.show()
```



8 Histograma

8.1 Função histograma

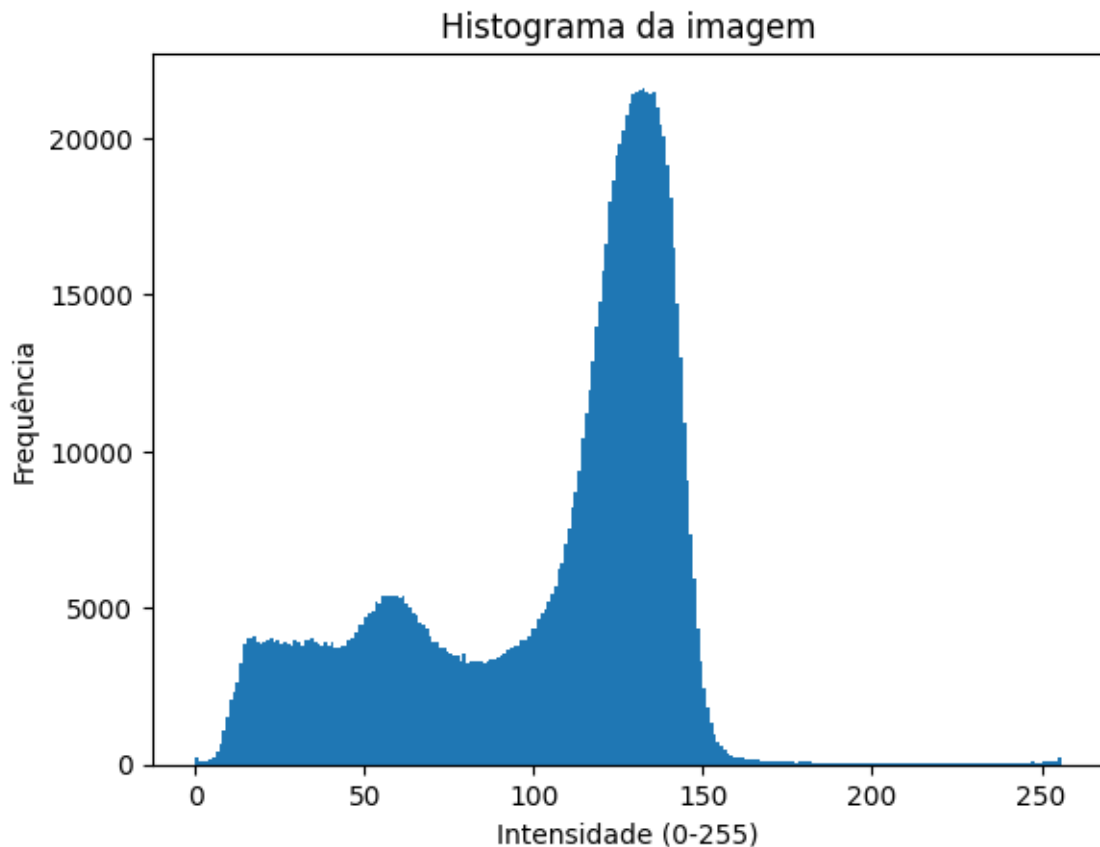
Faça uma função de nome `histograma` que recebe uma imagem em níveis de cinza f e faça um *plot* um histograma de frequência das intensidades dos *pixels* dessa imagem. Como a imagem f tem 256 níveis de intensidades, no seu histograma utilize 256 *bins*.

8.2 Visualização do histograma

Usando o código abaixo, visualize seu histograma.

Listing 4: Histograma de frequências

```
histograma (input_img)
```



O histograma ajuda a entender a distribuição de intensidades da imagem. Parafusos, porcas e arruelas geralmente aparecem como regiões mais escuras (valores menores) em contraste com o fundo. Você deve ver dois picos principais:

1. Um para o fundo (mais claro).
2. Outro para os objetos (mais escuros).

Quais as intensidades correspondentes a esses dois picos?

9 Limiarização (*Thresholding*)

9.1 Função limiar

Faça uma função de nome `limiar` que recebe uma imagem em níveis de cinza f e um inteiro t devolva a imagem binária b :

$$b(x, y) = \begin{cases} 255 & \text{se } f(x, y) \leq t \\ 0 & \text{caso contrário.} \end{cases}$$

9.2 Visualização de imagens binarizadas

Usando a função `limiar`, aplique limiares utilizando os valores `thresh_vals = [50, 60, 80, 100, 120, 140]` e apresente os resultados em um *plot* com as imagens binárias colocadas em uma grade 2×3 . Para tanto, utilize o código abaixo:

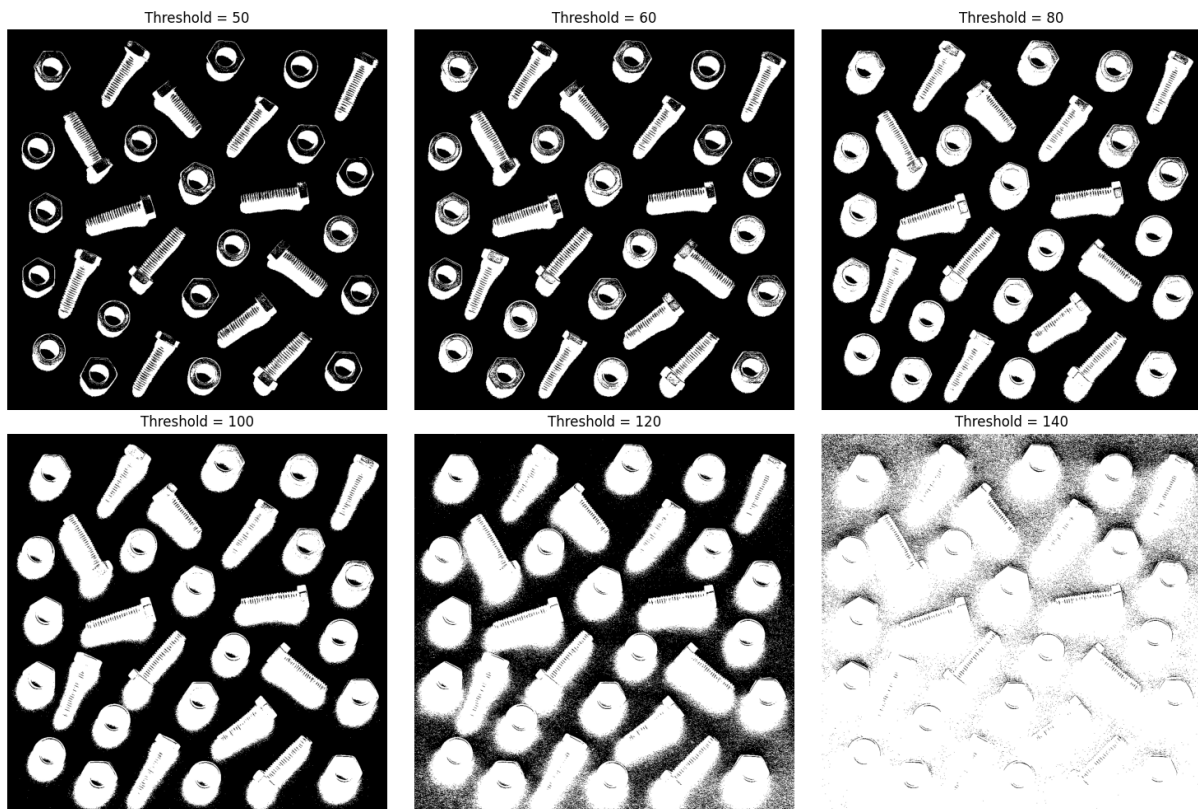
Listing 5: Aplicação de limiares

```
thresh_vals = [50, 60, 80, 100, 120, 140]

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.ravel()

for i, thresh in enumerate(thresh_vals):
    binary_img = limiar(input_img)
    axes[i].imshow(binary_img, cmap='gray')
    axes[i].set_title(f'Threshold = {thresh}')
    axes[i].axis('off')

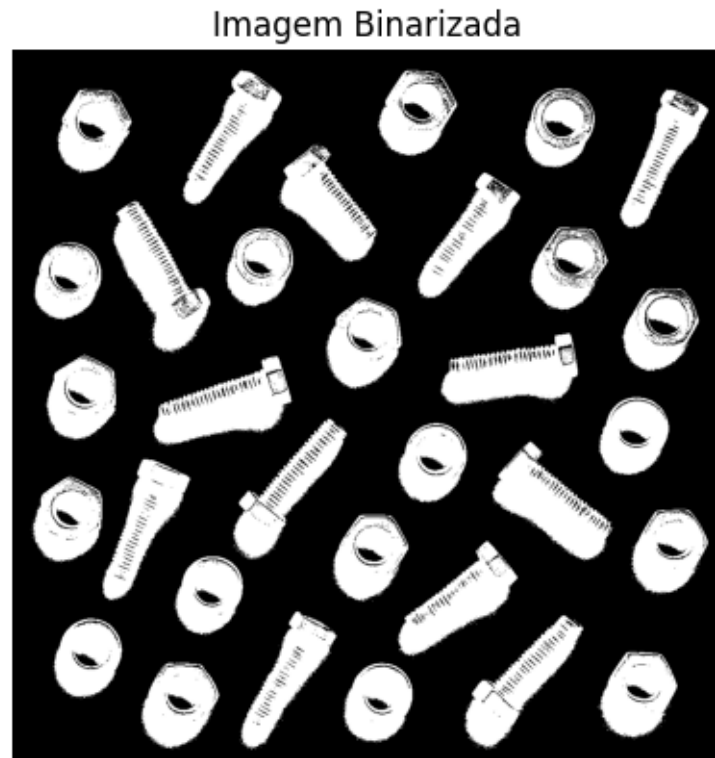
plt.tight_layout()
plt.show()
```



Se você observar atentamente as imagens binárias nos diferentes limiares, você perceberá que uma imagem adequada para detectar as peças (componentes conexos) é aquela que foi obtida com limiar igual a 80. Aplique a limiarização da imagem usando o código abaixo:

Listing 6: Aplicação de limiares

```
binary_img = limiar (input_img, 80)
# Exibindo a imagem binarizada
plt.imshow(binary_img, cmap="gray")
plt.title("Imagem Binarizada")
plt.axis("off")
plt.show()
```



9.3 Filtro de máxima

Se você observar atentamente a imagem binária `binary_img`, há pequenas regiões escuras dentro dos parafusos, porcas e arruelas. Para eliminar essas pequenas regiões escuras, vamos usar um filtro (não-linear) de máxima. Usando o *kernel* 5×5 , ele seleciona o valor de intensidade máxima na região dentro do *kernel*.

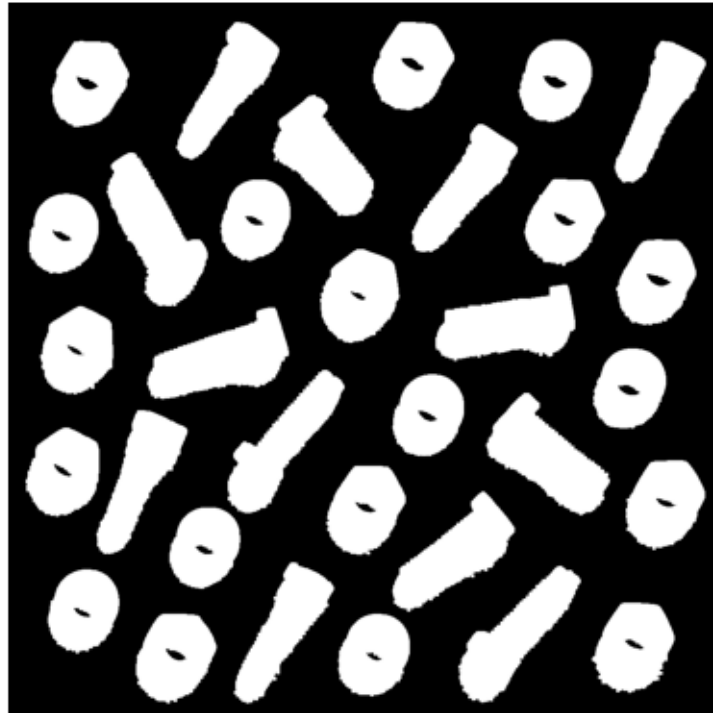
Escreva uma função de nome `filtro_maxima` que recebe uma imagem binária *b* e devolve a imagem *g* filtrada. Não se esqueça de fazer um *padding* de 2 *pixels* com zeros em torno da imagem.

Teste sua função com o código abaixo:

Listing 7: Filtragem

```
filtered_img = filtro_maxima (binary_img)
# Exibindo a imagem filtrada
plt.imshow(filtered_img, cmap="gray")
plt.title("Imagem Filtrada")
plt.axis("off")
plt.show()
```

Imagem Filtrada



10 Rotulagem de componentes conexos

Usando a estrutura de dados *union-find*, vamos rotular os componentes conexos da imagem `filtered_img`. O objetivo de rotular os componentes conexos é para contagem das peças na imagem binária. A estrutura de *union-find* é implementada por três funções: (1) `makeset`; (1) `find`; e (2) `union`. Use obrigatoriamente as implementações a seguir:

Listing 8: Union-Find data structure

```
def makeset (n):  
    '''  
    Inicializa a estrutura Union-Find para n elementos.  
  
    Cada elemento começa como seu proprio representante (ou "pai"),  
    formando n conjuntos distintos. O vetor 'rank' armazena a altura  
    estimada de cada arvore, utilizado para otimizar a operacao de uniao.  
  
    Parametros:  
        n (int): numero de elementos.  
  
    Retorna:  
        parent (list): vetor onde parent[i] representa o pai de i.  
        rank (list): vetor que guarda a altura estimada de cada arvore.  
    '''  
  
    # Complete aqui seu codigo  
  
    return parent, rank  
  
def find(i, parent):  
    '''  
    Encontra o representante (raiz) do conjunto ao qual o elemento i pertence.  
  
    Utiliza otimizacao por compressao de caminho (path compression),  
    fazendo com que cada no visitado durante a busca aponte diretamente  
    para a raiz do conjunto, o que acelera futuras operacoes.  
  
    Parametros:  
        i (int): elemento a ser localizado.  
        parent (list): vetor de pais.  
  
    Retorna:  
        int: representante do conjunto de i.  
    '''  
  
    # Complete aqui seu codigo  
  
    return parent[i]  
  
def union(i, j, parent, rank):  
    '''  
    Une os conjuntos que contem os elementos i e j.  
  
    Utiliza a heuristica de "union by rank": o conjunto com menor altura  
    e unido ao conjunto com maior altura. Se ambos tem a mesma altura,  
    um deles e escolhido arbitrariamente e sua altura e incrementada.  
  
    Parametros de Entrada:  
        i (int): primeiro elemento.  
        j (int): segundo elemento.  
        parent (list): vetor de pais.  
        rank (list): vetor de alturas estimadas das arvores.  
    Parametro de Saida:  
        parent (list): vetor de pais atualizado  
    '''  
  
    # Complete aqui seu codigo
```

10.1 Contagem de componentes conexos

Escreva uma função de nome `count_connected_components` que recebe uma imagem binária que (1) rotula os componentes conexos considerando conectividade 8; e (2) conta quantos componentes há na imagem. Para tanto use obrigatoriamente implementação da estrutura *Union-Find*.

Teste a sua função com o código abaixo:

Listing 9: Componentes conexos

```
num_components, parent_filtered = count_connected_components(filtered_img)  
print(f"Numero de componentes conexos: {num_components}")
```

Número de componentes conexos: 32

11 Detecção e contagem de parafusos, porcas e arruelas

Definition 1. Uma porca ou arruela é um componente que possui um buraco interno, ou seja, uma região com ao menos um pixel de fundo (ou seja, de intensidade 0) e completamente cercado por pixels de intensidade 255.

Essa definição é inspirada na estrutura física das porcas e arruelas reais, que geralmente possuem um furo central, ou seja, uma abertura em seu interior. No contexto de imagens binárias, esse “buraco” pode ser identificado como uma componente conexa na imagem complementar (onde os valores 0 e 255 são invertidos), que está totalmente contida dentro de uma componente maior da imagem original. Isso permite detectar porcas e arruelas com base em sua topologia, sem depender de forma geométrica exata. Buracos válidos podem ser identificados como componentes conexas na imagem complementar que estejam completamente contidas em algum objeto da imagem original.

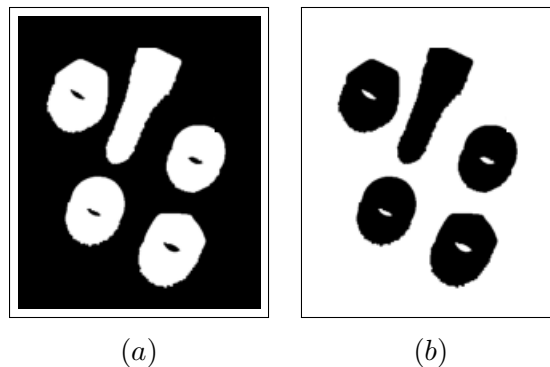


Figura 2: Exemplo de buracos em porcas e arruelas: (a) na imagem original. (b) na imagem complementar.

Observe os 4 buracos da imagem da Fig. 2(a) (não leve em conta o *background*). Esses buracos são os 4 componentes conexos da imagem da Fig. 2(b) (sem levar em conta o componente conexo obtido pela complementação do *background*). Observe que o parafuso na imagem da Fig. 2(a) não tem buracos.

11.1 Imagem complementar

Faça uma função de nome `complemento` que recebe uma imagem binária e devolve a sua imagem complementar.

Teste sua função com código abaixo:

Listing 10: Imagem complementar

```
# Calculate the complement of filtered_img
complement_filtered_img = complemento(filtered_img)

# Display the complement image
plt.imshow(complement_filtered_img, cmap="gray")
plt.title("Complement of Filtered Image")
plt.axis("off")
plt.show()
```

Complement of Filtered Image



11.2 Contagem e detecção de parafusos, porcas e arruelas

Para contar as porcas e arruelas basta contar os buracos na imagem filtrada ou equivalentemente contar os componentes conexos na imagem complementar.

Listing 11: Porcas e Arruelas

```
num_buracos, parent_buracos = count_connected_components(complement_filtered_img)
print(f"Numero de porcas e arruelas: {num_buracos-1}")
```

Foi subtraído 1 devido à componente conexa provindo do complemento do *background* da imagem original. Sim, o *background* da imagem original é o maior buraco da imagem complementar :-).

11.3 Calculando área dos buracos de cada componente conexo

Agora, vamos fazer uma tarefa difícil. Essa parte é a mais complicada do exercício. É desejado calcular a área (número de *pixels*) do buraco de cada componente conexo da imagem original. Por exemplo, a área do buraco de parafusos é zero (uma vez que parafusos não têm buracos). E cada porca e arruela tem um número associado diferente de zero que é a área do buraco no seu interior.

Componente	Área	Peça
1	0	Parafuso
2	23	Porca ou Arruela
3	25	Porca ou Arruela
4	19	Porca ou Arruela
5	35	Porca ou Arruela

Tabela 1: Componentes conexos da imagem da Fig. 2(a) e as respectivas áreas de seus buracos.

Note que o componente 1 tem área de buraco igual a zero e portanto é um parafuso. Os outros componentes têm área de buraco diferente de zero e portanto são porca ou arruela.

A primeira e segunda colunas da Tabela 1 pode ser guardado em uma variável tipo dicionário do *Python*: {1: 0, 2: 23, 3: 25, 4: 19, 5: 35}

Faça uma função de nome **area_buracos** que recebe uma imagem binária e devolve um dicionário contendo para cada componente conexo a área de seu buraco.



Dica

1. Primeiramente, calcule os componentes conexos da imagem de entrada
2. Depois, calcule a área dos componentes conexos da imagem complementar
3. Encontre o componente conexo de maior área da imagem complementar. Esse componente é o *background* da imagem de entrada. Você irá usar essa informação para detectar se um *pixel* que pertence a um buraco é de *background* ou não.
4. Agora, atribua para cada buraco (que não seja *background* da imagem de entrada), a sua área para o componente conexo correspondente.
5. Para cada *pixel* de buraco, verifique se existe um *pixel p* 8-vizinho que é *foreground* na imagem de entrada. Em caso positivo, encontre o correspondente componente conexo da imagem de entrada que contém esse *pixel p*. Atribua a esse componente a área do buraco.

Teste sua função para a imagem da Fig. 2(a) usando o seguinte código:

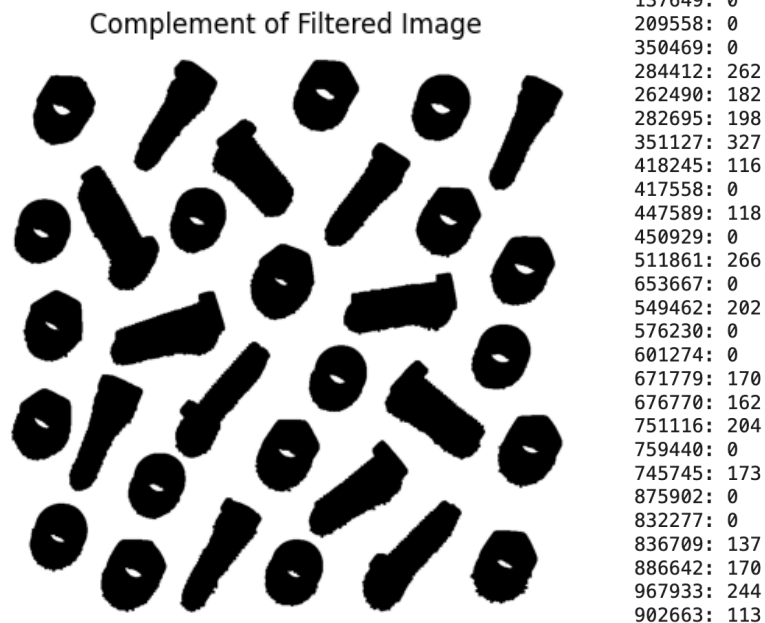
Listing 12: Área de Buracos dos Componentes Conexos

```
url = 'https://drive.google.com/file/d/1qBpQMeR-0YTgw-DJTf34CZgCnUVf4YCx/view?usp=sharing'
output_file_small = gdown.download(url, quiet=True, fuzzy=True)
print(f"File {output_file_small} downloaded")
small_input_img = load_image (output_file_small)
num_components, small_input_parent = count_connected_components(binary_small_input_img)
print(f"Numero de componentes conexos: {num_components}")
hole_areas_small = area_buracos(binary_small_input_img)
for key in hole_areas_small:
    print (f"{key}: {hole_areas_small[key]}")
```

Teste também usando o seguinte código:

Listing 13: Área de Buracos dos Componentes Conexos

```
hole_areas = area_buracos(filtered_img)
for key in hole_areas:
    print (f"{key}: {hole_areas[key]}")
```



11.4 Detecção de parafusos

Os parafusos são simplesmente os componentes conexos da imagem de entrada que não têm buracos, ou seja, que a área de buraco seja zero.

Faça uma função de nome `marca_conta_parafusos` que recebe como parâmetros uma imagem binária e devolve uma imagem binária somente com os parafusos e quantos parafusos há nessa imagem.

Teste sua função com o seguinte código:

Listing 14: Parafusos

```
# Calculate the complement of filtered_img
parafusos_img, num_parafusos = marca_conta_parafusos(filtered_img)

plt.imshow(parafusos_img, cmap="gray")
plt.title("Parafusos")
plt.axis("off")
plt.show()

print(f"Quantidade de parafusos: {num_parafusos}")
```

11.5 Separação de Porcas e Arruelas

Para obter uma imagem somente com as porcas e arruelas, basta diferença entre a imagem binária original e a imagem binária que contém somente os parafusos. Verifique se o resultado usando o código a seguir:

Listing 15: Porcas e Arruelas

```
subtracted_img = filtered_img - parafusos_img
plt.imshow(subtracted_img, cmap="gray")
plt.title("Porcas e Arruelas")
plt.axis("off")
plt.show()

num_components_subtracted, subtracted_parent = count_connected_components(subtracted_img)
print(f"Quantidade de porcas e arruelas: {num_components_subtracted}")
```

12 Entrega do EP1

Para entrega desse exercício, crie um *Google Colab Jupyter Notebook*. Esse *notebook* deve ser em formato de um relatório mostrando seu sistema de controle de qualidade automatizado, dividido em seções e mostrando de forma intercalada **textos** e todos os **códigos** pedidos desse EP (inclusive os códigos de teste). Ao final, faça o download do seu *Colab Notebook* em formato .ipynb e entregue-o na tarefa correspondente no e-disicplinas.